

Requirements, constraints & scope

Before you build a single chart, you have to answer four questions: what must it do, how well must it do it, what limits are you working inside, and where does the solution stop? Get these right and the whole data-analysis project has a spine.

What this key knowledge covers

This summary distils everything you need for **KK08 — Requirements, constraints & scope** in Data Analysis. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

Think of it as a renovation brief

Four words sound abstract until you map them onto something physical. So picture renovating a kitchen.

▣ Requirements = the job list

Functional = the things the kitchen must *do*: cook, store food, wash up. **Non-functional** = how *well*: the bench must take a hot pan, the lights must be bright enough to read a recipe.

For FreshStats: F "show a bar chart of sales by day" · NF "load in under 3 seconds on the canteen tablet".

▣ Constraints = the walls you can't move

Your **budget**, the size of the room, the plumbing already in the floor, building codes. You *must* design inside them — they're set by the world, not by you.

For FreshStats: C "no money for new software — use the school's Excel licence" · C "student purchase data must be de-identified (Privacy Act 1988)".

▣ Scope = the line around the worksite

Renovating the *kitchen* — not the bathroom, not the garden. The boundary tells everyone what's **in** and what's **out**, so nobody expects a new deck for the same price.

For FreshStats: in weekly sales dashboard · out live till integration, predicting next term's sales.

❑ Why bother being precise?

Vague briefs cause arguments later. "Make it nice" can't be ticked off; "labels must be readable on a 10-inch tablet" can. Precise requirements are *testable* — and in the exam, precise wording is what earns the marks.

Hold this analogy — we'll unpack each idea with FreshStats next.

Functional requirements — what it must do

A functional requirement describes a specific **action, feature or behaviour** the solution must perform. If you can picture the solution *doing* it — taking an input, processing it, producing an output — it's functional.

The reliable test: a functional requirement almost always starts with a **verb** — *calculate, display, sort, filter, import, export, search, validate*. It's something you could later **tick off as "built"**.

FreshStats functional requirements:

- **Import** the weekly sales CSV Ms Tran provides.
- **Calculate** total sales for each menu item.
- **Identify** the five best-selling items.
- **Display** a bar chart of sales by day of the week.
- **Filter** the dashboard by a selected week.
- **Export** the summary as a PDF for Ms Tran.

Functional requirements map cleanly onto **Input → Process → Output**:

If you can write it as a row of this table, it's a functional requirement.

Non-functional requirements — how well it must do it

A non-functional requirement describes a **quality or standard** the solution must meet. It doesn't add a new feature — it sets the bar for how the features behave. They're the "-ilities": usability, reliability, accessibility, maintainability, plus performance and security.

⚡ Performance

"Dashboard loads in < 3 seconds."

❑ Usability

"Ms Tran can find top sellers in one click."

& Accessibility

"Colours are distinguishable for colour-blind users."

❑ Security

"Only canteen staff can open the file."

❑ Reliability

"Charts still render if a day has no sales."



☐ Maintainability

"A new month's data can be added without rebuilding."

Functional vs non-functional — the one-question test

Students lose easy marks mixing these up. There's a single question that sorts almost every case.

☐ Ask: is it a thing it does, or how well it does it ?

Notice the last row: the *same feature* carries both. The action is functional; the standard attached to it is non-functional.

Constraints — the limits you must work inside

A constraint is a **restriction** on how the solution can be designed or developed — a boundary set by the outside world, not a feature you choose. You don't get to ignore them; you design *around* them.

☐ Economic

Money, time, licences, hardware budget.

"No budget for new software — use existing Excel; finished before end of term."

⚖ Legal

"Student purchase data must be de-identified — *Privacy Act 1988* (Cwlth)."

☐ Technical

Available tech, formats, compatibility.

"Must run on the canteen's old 10-inch tablet; data only comes as CSV."

☐ Social

People, culture, expectations.

"Canteen staff aren't tech experts — it must be simple to use."

☐ Usability

What users can reasonably operate.

"Must work with touch only — no mouse at the counter."

☐ Remember

Constraints come in five flavours: **economic, legal, technical, social, usability**. A deadline and a budget both sit under **economic**.

Scope — the boundary of the solution

Scope defines exactly **what is included (in scope)** and **what is excluded (out of scope)**. It draws a line around the project so everyone agrees where the solution ends — and what they *shouldn't* expect.

☐ In scope (FreshStats)

- Weekly sales dashboard from the provided CSV
- Bar chart of sales by weekday
- Top-5 selling items
- Food-waste summary (kg per week)
- Filter by week · export summary to PDF

☐ Out of scope (FreshStats)

- Live integration with the POS / till
- Predicting next term's sales
- A separate mobile app
- Online ordering for students
- Automatically ordering stock

How the four fit together

These aren't four separate lists — they describe one solution from four angles. Read a brief and you should be able to pull each one out.

Requirements say what the solution is and how good it is. **Constraints** say what you must work inside. **Scope** draws the line around the whole thing.

Seven traps that cost real marks

Every one of these is a mistake examiners see year after year. Knowing them is the cheapest mark on the page.

Write for the command word, structure with P·E·L

Marks follow the command word. Match your depth to the verb in the question, then structure each point so an examiner can find the marks fast.

The P·E·L spine

Every extended point can be built from three moves. Try it on this question:

Practice questions — weak vs strong, then self-mark

Try each in your head (or on paper) first. Reveal the weak answer to see the trap, the strong answer to see the ceiling, and the marking guide to award yourself marks. Your total tracks in the dock.

- 1 mark: a correct functional requirement with a FreshStats example.
- 1 mark: a correct non-functional requirement with a measurable FreshStats example.
- Full marks require the *distinction* (action vs quality/standard) to be explicit.
- 1 mark: a valid technical constraint with FreshStats detail.
- 1 mark: explaining it is an external limit the team must work within.
- 1 mark: a valid legal constraint (named law strengthens it).
- 1 mark: explaining it is imposed externally, not a feature/quality chosen by the team.
- 1 mark: identifies the request is out of the agreed scope.
- 1 mark: names this as scope creep.

- 1 mark: explains the consequence (deadline/resource/quality risk) and/or the value of a defined boundary.
- 1 mark: a re-write that is specific and measurable (a number or testable standard).
- 1 mark: it genuinely captures usability (not just speed).
- 1 mark: justification explicitly links measurability to being testable in evaluation.
- 1 mark each, (a)–(d): F, C, S, NF respectively.
- No part marks for a correct label with the wrong reason if working is shown.
- 1 mark: benefit of clear requirements (right features / testable standard).
- 1 mark: benefit of stating constraints (avoids impossible/illegal work).
- 1 mark: benefit of defined scope (prevents scope creep / protects deadline).
- 1 mark: a synthesising benefit (less rework, aligned expectations, basis for evaluation).

The KK08 cheat sheet

If you remember nothing else walking into the exam, remember this.

What the solution **does** — an action/feature. Starts with a verb; can be ticked off as built.

How **well** it does it — a measurable quality (the "-ilities").

External **limits** you must work inside: economic, legal, technical, social, usability.

The **boundary** — what's in and out. Stops scope creep.

Key terms

Term	Meaning
Lens	The question it answers
Functional	What must the solution <i>do</i> ?
Non-functional	How <i>well</i> must it do it?
Constraints	What <i>limits</i> bound the work?
Scope	Where does the solution <i>stop</i> ?

Common traps & misconceptions

Exam trap

1 Calling a non-functional requirement a constraint "Easy to read" is a quality you want → **NFR**. "Must use Excel because it's all we own" is an external limit → **constraint**. Ask: could the team have chosen differently? If no, it's a constraint.

Exam trap

2 Writing scope items as if they were requirements Scope is a **boundary** (in / out), not a list of actions. "Export to PDF" is a functional requirement; "the project does *not* include a mobile app" is a scope statement. Phrase scope as included / excluded.

Exam trap

3 Vague, untestable non-functional requirements "Fast", "good", "user-friendly" can't be measured or marked. Attach a number or standard: "loads in under 3 seconds".

Exam trap

4 Listing only functional requirements If a question says "requirements" and you only give actions, you've answered half of it. Always consider non-functional ones too.

Exam trap

5 Treating the deadline or budget as a requirement "Finish by the end of term" doesn't describe the solution — it limits the project. That's an economic constraint.

Exam trap

6 Confusing the data with a functional requirement "Sales data" is an input/source. A functional requirement is what the solution *does* with it. Lead with a verb: "calculates total sales".

Exam trap

7 Saying scope = the whole project Scope isn't everything — it's specifically the **boundary** of what's in and out. Don't define scope by restating the project title.

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Distinguish · 2 marks

Distinguish between a functional and a non-functional requirement, using one FreshStats example of each.

✗ A weak answer

A functional requirement is what the solution does and a non-functional requirement is how it works. For example the dashboard shows sales and it should be good. Why it loses marks: No clear difference is drawn, and 'should be good' is an opinion, not a measurable NFR. No FreshStats-specific functional example either.

✓ A strong answer

A **functional requirement** describes an action the solution must perform — e.g. "display a bar chart of FreshStats sales by weekday". A **non-functional requirement** describes a measurable quality of the solution — e.g. "the dashboard must load in under 3 seconds on the canteen tablet". The functional one is a feature you can tick off; the non-functional one is a standard that feature must meet.

How the marks are awarded

- 1 mark: a correct functional requirement with a FreshStats example.
- 1 mark: a correct non-functional requirement with a measurable FreshStats example.
- Full marks require the *distinction* (action vs quality/standard) to be explicit.

Q2. Explain · 4 marks

Describe one technical constraint and one legal constraint on the FreshStats solution. For each, explain why it is a constraint rather than a requirement.

✗ A weak answer

A technical constraint is the tablet and a legal constraint is privacy. These are constraints because they are important. Why it loses marks: Names the areas but gives no detail and no real explanation. 'Because they are important' is not a reason; it never explains the constraint-vs-requirement distinction.

✓ A strong answer

Technical: the dashboard must run on the canteen's old 10-inch tablet and the data is only supplied as CSV. This is a constraint because it is an external limit the team must design within — they cannot choose newer hardware or a different data format. **Legal:** any student-identifying purchase data must be de-identified under the *Privacy Act 1988* (Cwlth). This is a constraint, not a requirement, because it is imposed by law from outside the project rather than describing something the solution does or a quality it should have.

How the marks are awarded

- 1 mark: a valid technical constraint with FreshStats detail.
- 1 mark: explaining it is an external limit the team must work within.
- 1 mark: a valid legal constraint (named law strengthens it).
- 1 mark: explaining it is imposed externally, not a feature/quality chosen by the team.

Q3. Explain · 3 marks

Midway through, Ms Tran asks the team to also predict next term's sales. Explain, with reference to scope, why this is a problem.

✗ A weak answer

It is a problem because it is hard to do and they don't have time. Why it loses marks: Touches on time but never uses the concept of scope, which the question explicitly asks for. Misses the marks tied to scope/scope creep.

✓ A strong answer

Predicting next term's sales was defined as **out of scope** for FreshStats. Adding it after the boundary was agreed is **scope creep** — uncontrolled growth in what the project must deliver. This is a problem because it expands the work without extra time or resources, risking the deadline and the quality of the in-scope features. The defined scope lets the team professionally defer the request (e.g. to a future version) while still delivering what was agreed.

How the marks are awarded

- 1 mark: identifies the request is out of the agreed scope.
- 1 mark: names this as scope creep.
- 1 mark: explains the consequence (deadline/resource/quality risk) and/or the value of a defined boundary.

Q4. Justify · 3 marks

"The dashboard should be user-friendly." Re-write this as a measurable non-functional requirement and justify your wording.

✗ A weak answer

The dashboard should be very user-friendly and easy for everyone to use quickly. Why it loses marks: Still an opinion — 'very', 'easy', 'quickly' can't be measured or tested. No justification given.

✓ A strong answer

Re-written: "A canteen staff member can locate the top-selling item within one click and under 10 seconds, on the first attempt, without training." This is justified because it replaces the vague opinion 'user-friendly' with a **measurable, testable standard** — during evaluation the team can time the task and record success/failure, so it is clear whether the requirement has been met.

How the marks are awarded

- 1 mark: a re-write that is specific and measurable (a number or testable standard).
- 1 mark: it genuinely captures usability (not just speed).
- 1 mark: justification explicitly links measurability to being testable in evaluation.